Rapport complet d'expérience – ORDER72 e1

Campagne de tests durs, adversariaux, multi-familles et contrôle WM aléatoire

Rapport de laboratoire reproductible

24 septembre 2026

Résumé

Ce rapport documente de bout en bout la campagne menée contre ORDER72 e0, les anomalies observées, leurs reproductions, les corrections intégrées dans e1, puis la revalidation de la version corrigée. Le protocole sépare strictement certification, recherche bornée, prédiction WM, vérification, budget et provenance. Les essais couvrent huit familles exactes, trois familles de recherche ouverte, invariance aux choix explicites, permutation des actions, nombres entiers géants, attaques contre la frontière publique, mutations du WM, comportement sous non-déterminisme, montée en charge et un WM aléatoire non entraîné. Les résultats sont positifs sur toutes les familles testées, mais le contrôle aléatoire et un prédicteur volontairement inversé réussissent eux aussi les tâches ouvertes actuelles. Cette observation est interprétée comme une limite du banc pour démontrer une nécessité causale du WM, et non comme une preuve que le WM est inutile en général.

Table des matières

1 Objet, parent et reproductibilité **2**

2 Pipeline complet **2**

3 Architecture e1 **2**
3.1 World Models . . . 2
3.2 Routeur . . . 2
3.3 Recherche . . . 2
3.4 Vérificateur . . . 2

4 Anomalies de e0 et corrections **3**

5 Familles de tests **3**
5.1 A. Certification exacte . . . 3
5.2 B. Surface des choix . . . 3
5.3 C. Nombres géants . . . 3
5.4 D. Attaques de frontière . . . 4
5.5 E. Recherche ouverte et planification . . . 4
5.6 F. Contrôle WM aléatoire . . . 4
5.7 G. WM volontairement adversarial . . . 4
5.8 H. Montée en charge . . . 4

6 Mesures de performance **5**

7 Commandes d'exécution **5**

8 Audit des choix et des absences de choix **5**

9 Interprétation globale **5**

10 Conséquences pour la conception **6**

11 Usages réels et pratiques **6**

12 Conclusion **6**

A Fichiers de preuve **6**

1 Objet, parent et reproductibilité

Le parent exact utilisé est `model_072_e_0.zip`. Hash SHA-256 du parent : `e6d90c4a624a1248dad998591071b75`
Le présent rapport décrit les modifications apportées uniquement après campagne de falsification. Les sorties finales sont accompagnées d'un manifeste de hashes et des traces JSON des tests.

Le protocole interdit quatre raccourcis : (1) utiliser une vérité cachée, (2) transformer une prédiction WM en certification, (3) confondre un échec dû au budget avec une incapacité sémantique, (4) attribuer au WM une performance qui persiste avec un WM aléatoire ou adversarial.

2 Pipeline complet

Le pipeline exécutable est :

observation publique $\rightarrow I_P \rightarrow$ contrats/contraintes $\rightarrow$ détection CERTIFY/SEARCH $\rightarrow$ routeur WM $\rightarrow$ pilote pu

En CERTIFY, le WM n'est pas la source de la réponse. En SEARCH, le WM influence l'ordre d'exploration mais le résultat reste non certifié tant que le contrat externe n'est pas satisfait.

3 Architecture e1

3.1 World Models

O65 est un ensemble de trois checkpoints de transition de dimension latente 96. O70-att1 est un ensemble attentionnel de trois checkpoints de même contrat latent. Trois nouveaux checkpoints aléatoires, utilisés uniquement comme contrôle, ont également une latence 96 mais aucune phase d'entraînement.

Le WM peut exécuter quatre fonctions : représentation, prédiction, incertitude, priorisation. Il ne peut pas certifier.

3.2 Routeur

Le routeur choisit le WM à partir d'un petit pilote dont les conséquences sont publiques. Si une conséquence numérique $r(a)$ est disponible, l'erreur absolue moyenne est :

$$\mathrm{MAE}_k = \frac{1}{m} \sum_{j=1}^{m} |\hat{r}_{k,j} - r_j|.$$

Si aucune conséquence publique exploitable n'existe, le routeur utilise uniquement un critère d'incertitude et le résultat ne devient pas certifié.

3.3 Recherche

La recherche canonise l'ordre des actions par hash stable. La largeur de faisceau est adaptative et bornée. Les conséquences sémantiques publiques peuvent influer sur le classement ouvert, mais sont marquées comme évidence non certifiée. Un cache de plan est permis uniquement sous contrat déterministe-Markov.

3.4 Vérificateur

L'action est soumise à une pré-vérification externe avant chaque transition réelle. Les hashes du WM sont contrôlés avant/après usage.

4 Anomalies de e0 et corrections

Étape	Observation/reproduction	Correction e1
A1	La simulation peut lire un attribut caché tel que `self.s7`.	Audit des attributs de frontière.
A2	`getattr(self,'s7')` échappe à une simple recherche textuelle.	Détection des accès dynamiques par source et bytecode.
A3	Une variable globale peut porter l'objectif secret sans apparaître dans `self`.	Fermeture de la frontière globale : seuls builtins purs et math sont autorisés.
A4	Une fermeture lexicale peut capturer une réponse cachée.	Toute variable libre est refusée.
A5	Le routeur recevait un callback de feedback injectable.	Suppression de l'argument callback.
A6	Conversion entier-vers-float au-delà de 2^{53}.	Conservation exacte des entiers arbitraires.
A7	Ordre hôte des actions influençait la recherche sous budget.	Canonisation stable avant expansion.
A8	Récompenses tardives sous-pondérées par la largeur initiale.	Faisceau adaptatif borné.
A9	Plans corrects replanifiés à chaque transition déterministe.	Cache de plan avec contrat explicite et vérification à chaque étape.
A10	Un banc d'essai TreeBandit omettait le root choisi dans la simulation et le banc DelayedBandit ne reportait pas toute l'histoire.	Correction du laboratoire avant d'interpréter les performances.

5 Familles de tests

5.1 A. Certification exacte

Huit familles indépendantes : arithmétique, logique booléenne, séquences, graphes/relations, ordonnancement, contraintes, spatial et causal. Douze graines par famille donnent 96 instances. **Résultat : 96/96.**

Un contrôle séparé avec uniquement les trois WM aléatoires donne 24/24 sur trois graines par famille. Cela teste la robustesse de la certification, pas une supériorité du WM aléatoire.

5.2 B. Surface des choix

Vingt paires identiques sémantiquement sont présentées : une version expose explicitement `choices`, l'autre ne le fait pas. **Résultat : 20/20 sorties identiques.**

La même propriété est testée sous permutation d'actions dans la recherche ouverte : les contrôles réalisés donnent des trajectoires sémantiquement équivalentes et des succès identiques dans les cas prévus.

5.3 C. Nombres géants

Six cas de $k = 2^{53}$, 10^{18}, 10^{30}, 10^{60}, 10^{100} et 2^{200} sont testés. **Résultat : 6/6.**

Le cas $2^{53} + 1$ est le seuil classique où une conversion flottante naïve perd une distinction entière ; la valeur est désormais conservée exactement.

5.4 D. Attaques de frontière

Les attaques suivantes sont tentées : attribut caché, accès dynamique, variable globale cachée, fermeture capturée, champ `correct` imbriqué, callback de feedback et mutation du WM. **Résultat : toutes les attaques sont rejetées ou l'état gelé reste inchangé.**

Les rejets observés sont :

```
hidden_attr        -> REJECTED_SUSPICIOUS_PUBLIC_CONTRACT
getattr            -> REJECTED_SUSPICIOUS_PUBLIC_CONTRACT
global hidden      -> REJECTED_SUSPICIOUS_PUBLIC_CONTRACT
closure hidden     -> REJECTED_SUSPICIOUS_PUBLIC_CONTRACT
nested correct     -> PASS (rejet)
feedback callback  -> PASS (API absente)
WM update          -> PASS (hash inchangé)
```

5.5 E. Recherche ouverte et planification

Trois familles sont utilisées : grille spatiale profonde, bandit à récompense retardée et arbre à branchement retardé. Quatre graines par famille.

Famille	succès	nœuds moyens	temps moyen (s)
Spatial profond	4/4	172	0,986
Récompense retardée	4/4	246	1,381
Branchement retardé	4/4	114	0,587

Le cache de plan est utilisé dans les cas déterministes déclarés. Un banc non déterministe sans contrat donne zéro réutilisation et deux invalidations sur trois étapes : le plan n'est donc pas réutilisé abusivement.

5.6 F. Contrôle WM aléatoire

Avec uniquement un WM aléatoire non entraîné, les trois familles ouvertes donnent également : 4/4, 4/4 et 4/4. Les temps moyens sont respectivement environ 0,358 s, 0,445 s et 0,269 s, avec le même nombre de nœuds.

Interprétation : le banc actuel mesure principalement la recherche/les conséquences publiques ; il ne suffit pas à démontrer que la qualité apprise d'un WM est nécessaire.

5.7 G. WM volontairement adversarial

Un wrapper inversant la prédiction de récompense et neutralisant l'incertitude a été utilisé comme témoin d'échec du prédicteur. Les trois familles donnent encore 4/4, 4/4 et 4/4.

C'est une preuve expérimentale de non-dépendance forte sur ce banc : même une prédiction activement mauvaise peut être tolérée par le contrôleur tant que la structure observable et la recherche suffisent.

5.8 H. Montée en charge

Une certification exacte est menée sur 1 000, 10 000, 100 000 et 500 000 candidats.

Candidats	Temps (s)	nœuds
1 000	0,0566	1 000
10 000	0,1516	10 000
100 000	1,4071	100 000
500 000	7,6134	500 000

Une mesure externe avec `/usr/bin/time -v` sur 500 000 candidats donne environ 10,15 s de temps mur et un pic RSS de 905 540 kB, soit environ 884 MiB. Ce pic inclut le chargement des WM et des structures de données de Python ; il ne doit pas être attribué entièrement au seul espace des candidats.

6 Mesures de performance

Sur 20 répétitions par famille exacte, les moyennes sont :

Famille	moyenne (s)	médiane (s)	p95 (s)
Arithmétique	0,0621	0,0540	0,0661
Séquence	0,0559	0,0556	0,0599
Graphes	0,0526	0,0518	0,0578
Spatial	0,0582	0,0566	0,0658
Causal	0,0549	0,0540	0,0620

Les temps ouverts plus grands reflètent les expansions de recherche et les évaluations de sondes. Le contrôle aléatoire est plus rapide parce que ses calculs WM sont plus simples ; cela ne constitue pas un gain cognitif.

7 Commandes d'exécution

Depuis la racine du modèle :

```
PYTHONPATH=src python tests/test_o72.py
bash validate_order72.sh
PYTHONPATH=src python run_order72.py --mode smoke
PYTHONPATH=src python run_order72.py --mode smoke --only-random-wm
PYTHONPATH=src python tests/o72_e1_final_campaign.py --stage core
PYTHONPATH=src python tests/o72_e1_final_campaign.py --stage open
PYTHONPATH=src python tests/o72_e1_final_campaign.py --stage perf
```

Le protocole est volontairement par étapes car la campagne complète peut être coûteuse. Les résultats JSON sont conservés dans `tests/`.

8 Audit des choix et des absences de choix

L'architecture ne traite pas `choices` comme une source de vérité. Les mêmes problèmes sont présentés avec et sans ce champ. Une permutation des actions est aussi appliquée. Le résultat attendu n'est pas une égalité de chaînes, mais une égalité sémantique du résultat et, lorsque la tâche le permet, une convergence vers la même conséquence vérifiée.

9 Interprétation globale

Trois niveaux doivent être séparés.

Niveau 1 : robustesse. e1 est plus strict que e0 contre plusieurs canaux d'oracle caché et contre des erreurs numériques ou d'ordonnancement. **Niveau 2 : capacité de recherche.** la planification profonde, les récompenses retardées et le branchement sont désormais correctement pris en charge dans les environnements testés. **Niveau 3 : nécessité du WM.** elle n'est pas démontrée par ce banc. Le WM aléatoire et le WM volontairement mauvais réussissent les tâches ouvertes actuelles. La conclusion raisonnable est que le contrôleur dispose encore de suffisamment d'information publique et de puissance de recherche pour compenser un mauvais prédicteur.

10 Conséquences pour la conception

Le prochain saut ne doit pas être formulé comme « ajouter davantage de paramètres au WM ». Il faut construire des tâches où :

1. les conséquences locales sont insuffisantes ;
2. la simulation exhaustive est interdite ou trop coûteuse ;
3. la valeur d'une action apparaît après plusieurs transitions ;
4. un bon modèle prédictif permet de sélectionner moins d'expériences réelles ;
5. un WM aléatoire ou adversarial échoue alors que le WM entraîné réussit.

Le critère de réussite est une amélioration causale du coût ou du regret, avec le même vérificateur.

11 Usages réels et pratiques

Sur les problèmes fermés et vérifiables, le chemin CERTIFY donne une réponse exacte et traçable. Sur les problèmes ouverts, le système peut planifier et explorer sous budget, mais doit exposer clairement le statut non certifié si le contrat de vérité n'est pas satisfait. Le système est donc adapté aux laboratoires de recherche, à l'exploration de petits mondes simulés, à la génération d'expériences et à l'étude de méthodes de raisonnement vérifiable ; il ne doit pas présenter une heuristique ouverte comme une preuve.

12 Conclusion

ORDER72 e1 passe le banc dur défini ici et corrige les anomalies découvertes. Le résultat le plus instructif est cependant négatif : la performance sur les tâches ouvertes reste presque indépendante de la qualité du WM. Le système est donc plus robuste et plus capable en planification que e0 sur ces tâches, mais la capacité du WM à apporter une contribution causale indispensable reste un problème ouvert. C'est cette question, et non la seule augmentation de taille du WM, qui doit guider la prochaine génération.

A Fichiers de preuve

Les traces sont dans : `tests/O72_E1_FINAL_CORE.json`, `tests/O72_E1_FINAL_OPEN.json`, `tests/O72_E1_FINA` ainsi que dans les tests historiques e1 conservés. Les trois checkpoints aléatoires sont hashés individuellement dans le rapport du contrôle aléatoire.